

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Tejaswi Reddy K	Reg. No.:	192371036
Section / Batch:	CS_BIOSCI	Date:	25-08-26

Instructions to Students

- Answer both questions carefully. Each question carries 50 marks. Total: 100 marks.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Description	Questions	Total Marks
Comparative Analysis	Focus on Comparison of different Algorithms using dynamic programming and greedy strategies.	Q1 – Q2	100

SECTION A – Comparative Analysis

Q1. Assembly Line Scheduling vs Word Break (DP Comparison)
 Given scheduling and string segmentation problems, solve using DP. Compare recurrence relations and decision-making. Analyze differences in constraints. Evaluate complexity. Discuss scalability. Generate insights on DP versatility.

SOLUTION –

1. Definition of Assembly Line Scheduling and Word Break

A. Assembly Line Scheduling

The **Assembly Line Scheduling Problem** determines the minimum time required to manufacture a product by passing it through a sequence of stations arranged on two assembly lines.

Each assembly line has n stations. At every station, a product can either:

1. Stay on the same assembly line, or
2. Switch to the other assembly line.

Switching requires additional **transfer time**.

Example

Consider two assembly lines with 4 stations:

```
Line 1:  A1 ---- A2 ---- A3 ---- A4
          |         |         |         |
          |         |         |         |
Line 2:  B1 ---- B2 ---- B3 ---- B4
```

Processing times:

Station	Line 1	Line 2
1	4	2
2	5	10
3	3	1
4	2	4

Transfer times:

Transfer	Time
Line 1 → Line 2	7
Line 2 → Line 1	9

Entry and exit times are also considered.

The objective is to find the **minimum total time** required to move the product from the beginning to the end.

B. Word Break

The **Word Break Problem** determines whether a given string can be completely divided into valid words from a given dictionary.

Example

String:
 "catsanddog"

Dictionary:
 {"cat", "cats", "and", "sand", "dog"}

The string can be segmented as:

"cats" + "and" + "dog"

Therefore:

Output: TRUE

2. Pseudo Code –

Assembly Line –

```
ASSEMBLY_LINE(a, t, e, x, n)
f1[0] = e1 + a1[0]
f2[0] = e2 + a2[0]
for j = 1 to n-1:
    f1[j] = min(
        f1[j-1] + a1[j],
        f2[j-1] + t2[j-1] + a1[j]
    )
    f2[j] = min(
        f2[j-1] + a2[j],
        f1[j-1] + t1[j-1] + a2[j]
    )
answer = min(
    f1[n-1] + x1,
    f2[n-1] + x2
)
return answer
```

Word Break –

WORD_BREAK(string, dictionary)

n = length(string)

dp[0] = TRUE

for i = 1 to n:

 dp[i] = FALSE

 for j = 0 to i-1:

 if dp[j] == TRUE

 and substring(j, i) exists in dictionary:

 dp[i] = TRUE

 break

return dp[n]

3. Assembly Line Scheduling State

The state represents the **minimum time required to reach a particular station on a particular assembly line.**

f1[j] = minimum time to reach station j on Line 1

f2[j] = minimum time to reach station j on Line 2

B. Word Break State

For a string:

"catsanddog"

we can represent the state as:

Index:	0	1	2	3	4	5	6	7	8	9	10
		---		---		---		---		---	
String:	c	a	t	s	a	n	d	d	o	g	

The DP state is:

$dp[i]$ = whether $s[0...i-1]$ can be segmented

4. Assembly Line Scheduling Recurrence

For Line 1:

$f1[j] = \min\{f1[j-1] + a1[j], f2[j-1] + a1[j]\}$

For Line 2:

$f2[j] = \min\{f2[j-1] + a2[j], f1[j-1] + a2[j]\}$

Finally:

Answer = $\min(f1[n-1] + x1, f2[n-1] + x2)$

B. Word Break Recurrence

Let:

$dp[i]$ = whether the first i characters can be segmented

Then:

$dp[i] = \bigvee_{j=0}^{i-1} (dp[j] \wedge \text{dictionary}(s[j:i]))$

5. Implementation in Saveetha Compiler

The screenshot shows the SIMATS Online Programming Lab interface. The top navigation bar includes the SIMATS Engineering logo, the lab name, and a user profile. The main content area is titled "CO4-AT3-Comparative Analysis" and features a "Language" dropdown set to "Python". A "QUESTIONS" sidebar on the left lists two questions, with the second one, "Greedy Technique", selected. The main panel displays the question details for "Greedy Technique", which includes a sub-header "Assembly Line Scheduling vs Word Break (DP Comparison)" and a description of the problem. Below the question, there is a "Your Code" section with a dark-themed code editor containing Python code for the assembly line scheduling problem. To the right of the code editor is an "Input" field and an "Output" field. The "Output" field shows the result of the code execution: "Assembly Line Scheduling - Minimum Time: 10, Word Break - Can segment 'tactacode': True". At the bottom right, there is a watermark for "Activate Windows" and a link to "Go to Settings to activate Windows".

```

1 // C++ program for Assembly Line Scheduling
2
3 #include <iostream>
4
5 using namespace std;
6
7 // Function to find the minimum time required to complete the assembly line
8 int findMinTime(int n, int s1[], int s2[], int t1[], int t2[])
9 {
10     // Create an array to store the minimum time required to complete the assembly line
11     int min_time = 0;
12
13     // Iterate over all possible splitting positions
14     for (int i = 0; i < n; i++)
15     {
16         // Calculate the time required to complete the assembly line at position i
17         int time = 0;
18         for (int j = 0; j < i; j++)
19             time += t1[j];
20         for (int j = i; j < n; j++)
21             time += t2[j];
22
23         // Update the minimum time
24         min_time = min(min_time, time);
25     }
26
27     return min_time;
28 }
29
30 // Driver code
31 int main()
32 {
33     // Example Input
34     int n = 5;
35     int s1[] = {10, 12, 20, 15, 10};
36     int s2[] = {12, 15, 10, 15, 10};
37     int t1[] = {10, 12, 20, 15, 10};
38     int t2[] = {12, 15, 10, 15, 10};
39
40     // Find the minimum time required to complete the assembly line
41     int min_time = findMinTime(n, s1, s2, t1, t2);
42
43     // Print the minimum time
44     cout << "Minimum Time Required = " << min_time << endl;
45
46     return 0;
47 }

```

```

1 // C++ program for Word Break
2
3 #include <iostream>
4
5 using namespace std;
6
7 // Function to find the minimum time required to complete the assembly line
8 int findMinTime(int n, int s1[], int s2[], int t1[], int t2[])
9 {
10     // Create an array to store the minimum time required to complete the assembly line
11     int min_time = 0;
12
13     // Iterate over all possible splitting positions
14     for (int i = 0; i < n; i++)
15     {
16         // Calculate the time required to complete the assembly line at position i
17         int time = 0;
18         for (int j = 0; j < i; j++)
19             time += t1[j];
20         for (int j = i; j < n; j++)
21             time += t2[j];
22
23         // Update the minimum time
24         min_time = min(min_time, time);
25     }
26
27     return min_time;
28 }
29
30 // Driver code
31 int main()
32 {
33     // Example Input
34     int n = 5;
35     int s1[] = {10, 12, 20, 15, 10};
36     int s2[] = {12, 15, 10, 15, 10};
37     int t1[] = {10, 12, 20, 15, 10};
38     int t2[] = {12, 15, 10, 15, 10};
39
40     // Find the minimum time required to complete the assembly line
41     int min_time = findMinTime(n, s1, s2, t1, t2);
42
43     // Print the minimum time
44     cout << "Minimum Time Required = " << min_time << endl;
45
46     return 0;
47 }

```

6. Time Complexity Analysis

Assembly Line Scheduling

There are n stations.

At every station, only **two choices** are considered:

Stay

Switch

Therefore, each station requires constant work.

$T(n) = O(n)$

Time Complexity:

$O(n)$

This makes Assembly Line Scheduling highly efficient.

Word Break

For a string of length n , we consider all possible splitting positions.

There are approximately $O(n^2)$ possible substrings.

If dictionary lookup is assumed to take $O(1)$ using a hash set, the DP computation is approximately: $O(n^2)$

Therefore: $O(n^2)$

However, if dictionary lookup or substring comparison takes additional time, the practical complexity can be higher.

7. Space Complexity

Assembly Line Scheduling

Using two arrays:

$f1[]$

$f2[]$

requires:

$O(n)$

space.

Therefore:

$O(n)$

However, because only the previous station's values are needed, the algorithm can be optimized to:

$O(1)$

auxiliary space.

Word Break

The DP array contains $n + 1$ states:

$dp[0], dp[1], \dots, dp[n]$

Therefore:

$O(n)$

space.

If actual substrings are created during implementation, additional temporary memory may be used.

8. Comparison of the Two DP Formulations

Feature	Assembly Line Scheduling	Word Break
Problem type	Scheduling/optimization	String segmentation
Input	Assembly lines and station times	String + dictionary
DP state	Minimum time to reach station	Whether prefix can be segmented
State type	Numeric	Boolean
Objective	Minimize total time	Determine possible segmentation
Main decision	Stay or switch line	Choose valid previous word
Recurrence	Minimum of two alternatives	OR over possible split points
Number of states	$O(n)$	$O(n)$
Transitions per state	Constant	Up to $O(n)$
Time complexity	$O(n)$	$O(n^2)^*$
Space complexity	$O(n) / O(1)$ optimized	$O(n)$
Input constraints	Station sequence and transfer costs	String length and dictionary size
Scalability	Very good	Depends on string and dictionary size

9. Similarities Between Assembly Line Scheduling and Word Break

- Both contain overlapping subproblems
- Both use optimal substructure
- Both store intermediate results
- Both use recurrence relations
- Both avoid exponential brute force

10. Differences in State Definition and Transitions

Aspect	Assembly Line Scheduling	Word Break
State meaning	Minimum cost/time	Feasibility of segmentation
State variable	Station + line	String prefix
DP value	Integer	Boolean
Decision	Stay or switch	Split at a valid word
Transition count	2	Up to i
Optimization type	Minimization	Boolean feasibility
Dependencies	Previous station	All previous prefix positions
Main constraint	Processing + transfer times	Dictionary membership

11. DP State, Transition, Implementation and Complexity Comparison

Category	Assembly Line Scheduling	Word Break
DP State	$f[\text{line}][\text{station}]$ = minimum time to reach station	$dp[i]$ = whether prefix of length i can be segmented
Initial State	Entry time + first station time	$dp[0] = \text{TRUE}$
Transition	Stay on line or switch lines	Try every previous split point
Recurrence	$\min(\text{stay}, \text{switch})$	OR(valid previous prefix + dictionary word)
Decision-making	Choose minimum-cost path	Find any valid segmentation
Implementation	Arrays for two lines	Boolean DP array
Time	$O(n)$	$O(n^2)^*$
Space	$O(n)$, reducible to $O(1)$	$O(n)$
Scalability	Excellent for many stations	Good, but grows quadratically with string length
Main limitation	More complex constraints require additional states	Large dictionaries/long strings can increase lookup cost

Q2. Dijkstra vs Bellman-Ford (Performance Comparison)
Given weighted graphs, compare Dijkstra and Bellman-Ford algorithms. Analyze efficiency and limitations with negative edges. Evaluate computational complexity. Generate insights on algorithm selection in networks.

SOLUTION –

1. The **Word Break Problem** is a problem in which a given string is divided into one or more valid words from a given **dictionary**. The objective is to determine whether the entire string can be segmented into dictionary words.

Ex - Given:

String: "applepie"

Dictionary: {"apple", "pie", "app", "le"}

The string can be divided as:

"apple" + "pie"

Since both "apple" and "pie" are present in the dictionary, the answer is:

Output: TRUE

The **Longest Palindromic Subsequence (LPS)** problem is the problem of finding the **longest subsequence of a string that is a palindrome**. A **palindrome** is a sequence that reads the same forward and backward. A **subsequence** does not require the selected characters to be adjacent; characters can be skipped while maintaining their original order.

Ex - Given:

String = "BBABCBCAB"

One longest palindromic subsequence is:

"BABCBAB"

It reads the same in both directions:

B A B C B C B

B C B C B A B ← reverse

Its length is: 7

Therefore:

LPS = "BABCBAB" (one possible LPS)

Length = 7

There can be **more than one valid longest palindromic subsequence**.

2. Dijkstra Pseudocode -

```
DIJKSTRA(G, source)
for each vertex v in G:
    distance[v] = ∞
    parent[v] = NULL
distance[source] = 0
Create a priority queue Q
Insert source into Q
while Q is not empty:
    u = vertex with minimum distance
    for each edge (u, v) with weight w:
        if distance[u] + w < distance[v]:
            distance[v] = distance[u] + w
            parent[v] = u
    update Q
return distance
```

Bellman-Ford Pseudocode -

```
BELLMAN-FORD(G, source)
for each vertex v:
    distance[v] = ∞
    parent[v] = NULL
distance[source] = 0
for i = 1 to |V| - 1:
    for each edge (u, v) with weight w:
        if distance[u] + w < distance[v]:
            distance[v] = distance[u] + w
            parent[v] = u
for each edge (u, v) with weight w:
    if distance[u] + w < distance[v]:
        return "Negative-weight cycle exists"
return distance
```

3. Dijkstra State

dist[v] = shortest distance currently known
from source to vertex v

Bellman-Ford State

$D[k][v]$ = shortest distance from source
to vertex v using at most k edges

4. Dijkstra Transition

For an edge (u, v) with weight w :

$$D[v] = \min(D[v], D[u] + w)$$

Bellman-Ford Recurrence

Let:

$D_k(v)$

represent the shortest distance from the source to vertex v using at most k edges.

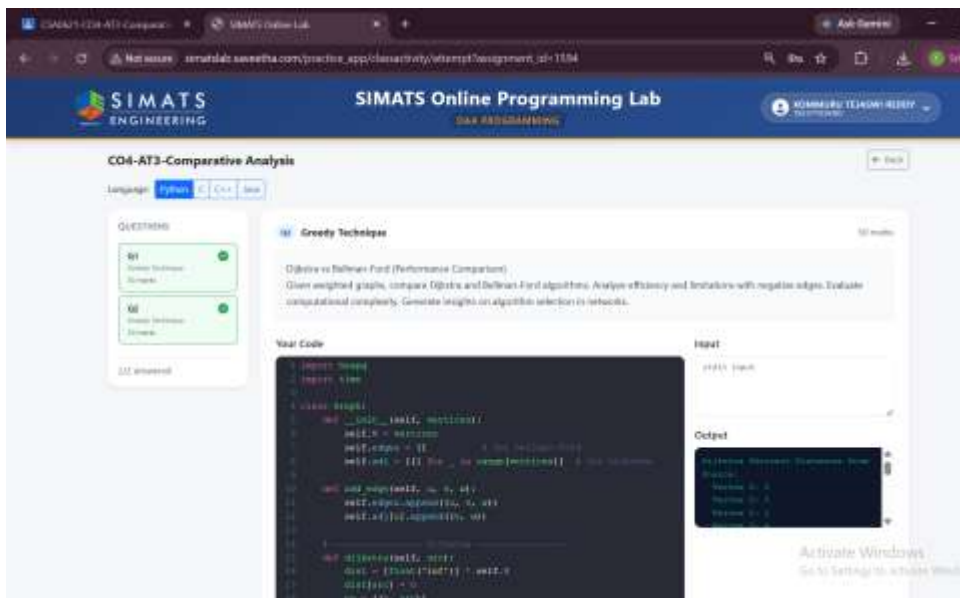
Then:

$$D_k(v) = \min(D_{k-1}(v), \min_{(u,v) \in E} \{D_{k-1}(u) + w(u,v)\})$$

In simpler form:

$$D[v] = \min(D[v], D[u] + w(u,v))$$

5. Implementation in Saveetha Compiler



CSAM21-CD4-AT3-Comparat... SIMATS Online Lab

Not secure simatslab.savertha.com/practice_app/classactivity/attempt/assignment_id=1594

Ask Gemini

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

```
11 while pos
12     u = heapq.heappop(hq)
13     if visited[u]:
14         continue
15     visited[u] = True
16
17     for v, w in self.adjlist:
18         if not visited[v] and dist[u] + w < dist[v]:
19             dist[v] = dist[u] + w
20             heapq.heappush(hq, (dist[v], v))
21
22     return dist
23
24 # ----- Bellman-Ford -----
25 def bellman_ford(self, src):
26     dist = {node: float('inf') for node in self.V}
27     dist[src] = 0
28
29     # Relax all edges V-1 times
30     for i in range(1, V - 1):
31         for u, v, w in self.edges:
32             if dist[u] + float('inf') < dist[v] + w < dist[v]:
33                 dist[v] = dist[u] + w
34
35     # Check for negative-weight cycles
36     for u, v, w in self.edges:
37         if dist[u] + float('inf') < dist[v] + w < dist[v]:
38             print("Graph contains a negative weight cycle!")
39             return None
40
41     return dist
42
43 # ----- Driver Code -----
44 if __name__ == '__main__':
45     V = 5
46     g = Graph(V)
47
48     # Sample edges with non-negative weights
49     edges_positive = [
50         (0, 1, 4), (0, 2, 1), (2, 3, 3),
51         (1, 3, 1), (1, 4, 3), (3, 4, 3)
52     ]
53     for u, v, w in edges_positive:
54         g.add_edge(u, v, w)
55
56     src = 0
57
58     # ----- Dijkstra -----
59     start = time.time()
60     dijkstra_dist = g.dijkstra(src)
61     dijkstra_time = time.time() - start
62     print_distances("Dijkstra", dijkstra_dist)
63     print(f"Dijkstra Execution Time: {dijkstra_time:.3f} sec")
64
65     # ----- Bellman-Ford -----
66     start = time.time()
67     bellman_dist = g.bellman_ford(src)
68     bellman_time = time.time() - start
69     print_distances("Bellman-Ford", bellman_dist)
```

Activate Windows
Go to Settings to activate Windows

CSAM21-CD4-AT3-Comparat... SIMATS Online Lab

Not secure simatslab.savertha.com/practice_app/classactivity/attempt/assignment_id=1594

Ask Gemini

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KOMMURU TEJASWI REDDY

```
11 def print_distances(name, dist):
12     print(f"{name} Shortest Distances from Source:")
13     for u, d in enumerate(dist):
14         print(f"Vertex {u} is {d} (is {float('inf')} else 'inf')")
15
16 # ----- Driver Code -----
17 if __name__ == '__main__':
18     V = 5
19     g = Graph(V)
20
21     # Sample edges with non-negative weights
22     edges_positive = [
23         (0, 1, 4), (0, 2, 1), (2, 3, 3),
24         (1, 3, 1), (1, 4, 3), (3, 4, 3)
25     ]
26     for u, v, w in edges_positive:
27         g.add_edge(u, v, w)
28
29     src = 0
30
31     # ----- Dijkstra -----
32     start = time.time()
33     dijkstra_dist = g.dijkstra(src)
34     dijkstra_time = time.time() - start
35     print_distances("Dijkstra", dijkstra_dist)
36     print(f"Dijkstra Execution Time: {dijkstra_time:.3f} sec")
37
38     # ----- Bellman-Ford -----
39     start = time.time()
40     bellman_dist = g.bellman_ford(src)
41     bellman_time = time.time() - start
42     print_distances("Bellman-Ford", bellman_dist)
```

Activate Windows
Go to Settings to activate Windows

```
11 # Bellman-Ford
12 start = time.time()
13 bellman_dist = q.bellman_ford(dist)
14 bellman_time = time.time() - start
15 print_distances("bellman-ford", bellman_dist)
16 print("Bellman-Ford Execution time: {0} sec".format(bellman_time))
17
18 # Script ends with a negative edge, all other fails here
19 print("\n--- Testing with Negative Edge Weights ---")
20 g = Graph()
21 edges_negative = 1
22 [(0, 1, 4), (0, 2, 3), (2, 1, -1),
23  (1, 3, 1), (2, 4, 5), (3, 4, 1)]
24 )
25 for i, u, v, w in edges_negative:
26     g.add_edge(u, v, w)
27
28 bellman_dist = q.bellman_ford(dist)
29 print_distances("bellman-ford (with negative edge)", bellman_dist)
30 print("\nNote: Dijkstra would give INCORRECT results here")
31 print("because it assumes all edge weights are non-negative.")
32
33 # Complexity Summary
34 print("\n--- Computational Complexity Summary ---")
35 print("Dijkstra (using Min-Heap): O((V + E) log V)")
36 print("Bellman-Ford: O(V * E)")
37 print("\n--- Insights ---")
38 print("1. Dijkstra is faster but fails with negative edge weights.")
39 print("2. Bellman-Ford is slower but handles negative weights")
40 print("   and can detect negative weight cycles.")
41 print("3. Use Dijkstra for large graphs with non-negative weights")
42 print("   (e.g., GPS/route networks).")
```

6. Time Complexity Analysis

Let: V = number of vertices , E = number of edges

Dijkstra

Using an adjacency matrix and simple array:

$O(V^2)$

Using an adjacency list and binary heap:

$O((V+E)\log V)$

For sparse graphs where:

$E \approx V$

the complexity becomes approximately:

$O(V\log V)$

Bellman-Ford

Bellman-Ford relaxes every edge $V - 1$ times.

Therefore:

$O(VE)$

This is considerably slower than an efficient implementation of Dijkstra for many graphs.

7. Space Complexity

Dijkstra

Typical storage includes:

Distance array $\rightarrow O(V)$

Visited array $\rightarrow O(V)$

Priority queue $\rightarrow O(V)$

Graph $\rightarrow O(V + E)$

Therefore, with an adjacency-list representation:

$O(V+E)$

Bellman-Ford

Storage includes:

Distance array $\rightarrow O(V)$

Parent array $\rightarrow O(V)$

Edge list $\rightarrow O(E)$

Therefore:

$O(V+E)$

If only the algorithm's auxiliary space is considered, excluding the graph representation:

$O(V)$

for both algorithms.

8. Comparison of the Two Algorithms

Feature	Dijkstra	Bellman-Ford
Algorithm type	Greedy	Dynamic-programming-style relaxation
Purpose	Single-source shortest path	Single-source shortest path
Negative edges	Not supported	Supported
Negative cycle detection	No	Yes
Basic transition	Relaxation	Repeated relaxation
Main strategy	Select minimum-distance vertex	Relax every edge repeatedly
Time complexity	$O(V^2)$ or $O((V+E) \log V)$	$O(VE)$
Space complexity	$O(V+E)$	$O(V+E)$
Speed	Faster	Slower
Best for	Non-negative weighted graphs	Graphs with negative edges
Implementation	Relatively more complex with priority queue	Relatively straightforward
Early termination	Possible	Possible if no update occurs
Network suitability	High when weights are non-negative	Useful when negative costs are possible

9. Similarities Between Dijkstra and Bellman-Ford

Both algorithms have several similarities:

- Single-source shortest path:**
Both calculate shortest paths from one source vertex to other vertices.
- Weighted graphs:**
Both operate on weighted graphs.
- Distance array:**
Both maintain a distance value for each vertex.
- Edge relaxation:**
Both use the fundamental operation: $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w)$
- Path reconstruction:**
Both can maintain a predecessor/parent array to reconstruct the shortest path.
- Network applications:**
Both can be used for routing and shortest-path problems.

10. Differences in State Definition and Transitions

Aspect	Dijkstra	Bellman-Ford
State	Current shortest tentative distance	Shortest distance after repeated edge relaxation
State interpretation	Some vertices become permanently finalized	Distances progressively improve
Selection	Selects minimum-distance unvisited vertex	No vertex selection required
Transition	Relax edges from selected vertex	Relax every edge
Number of major iterations	Up to V	$V - 1$
Negative edges	Cannot safely handle them	Can handle them
Negative cycle	Cannot detect reliably	Can detect reachable negative cycles

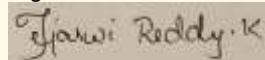
11. Performance Comparison.

Parameter	Dijkstra	Bellman-Ford
Sparse graph with binary heap	$O((V+E) \log V)$	$O(VE)$
Dense graph	$O(V^2)$	$O(VE)$
Negative edges	☐	☐
Negative-cycle detection	☐	☐
Practical speed	High	Lower
Reliability with arbitrary edge weights	Limited	High

Code of Conduct

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Mark Evaluation:

Questions	Selection of Studies/Parameters (20)	Comparison & Differentiation (25)	Critical Evaluation (25)	Synthesis & Insight Generation (20)	Clarity (10)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total						

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____